

# noted



**iscte**  
UNIVERSITY  
INSTITUTE  
OF LISBON

**Engineering of Intelligent Models**

**FINAL DELIVERY**

**USER MANUAL**

**NOTED PLATFORM**

## Table of Contents

|                                                         |    |
|---------------------------------------------------------|----|
| Page 1: Your First Project .....                        | 5  |
| The noted interface .....                               | 5  |
| Step 1: Create a project .....                          | 5  |
| Step 2: Create a notebook .....                         | 5  |
| Step 3: Create a Python environment .....               | 6  |
| Step 4: Install packages .....                          | 6  |
| Step 5: Connect the environment to your notebook .....  | 6  |
| Step 6: Run your first cell .....                       | 7  |
| Where to go next .....                                  | 7  |
| Page 2: Configuring Your Experiment .....               | 8  |
| What is Hydra configuration? .....                      | 8  |
| Step 1: Create the config directory structure .....     | 8  |
| Step 2: Write the config files .....                    | 9  |
| Step 3: Enable Hydra view on the config folder .....    | 10 |
| Step 4: Open the Configuration Composer .....           | 10 |
| Step 5: Apply selections to the notebook .....          | 11 |
| Step 6: The baseline badge in the notebook bar .....    | 12 |
| Step 7: Use the config in your notebook .....           | 12 |
| Step 8: Per-run Hydra bundle logging .....              | 13 |
| Step 9: Travel back in time .....                       | 13 |
| Step 10: Creating new baselines from past runs .....    | 14 |
| Step 11: Run your experiment with MLflow tracking ..... | 14 |
| Step 12: Change parameters and re-run .....             | 15 |
| Where to go next .....                                  | 15 |
| Page 3: Running Your Experiment .....                   | 16 |
| What is MLflow? .....                                   | 16 |
| Step 1: Define a run in the Run Manager .....           | 16 |
| Step 2: Execute the run .....                           | 16 |
| Step 3: Monitor live metrics during training .....      | 17 |
| Step 4: Explore results in the Experiments tree .....   | 17 |
| Step 5: The Lineage view .....                          | 18 |
| Step 6: Compare two runs .....                          | 19 |
| Step 7: Ask the Assistant about a run .....             | 20 |
| Step 8: Reproduce or iterate from a past run .....      | 20 |
| Where to go next .....                                  | 20 |

|                                                               |    |
|---------------------------------------------------------------|----|
| Page 4: From Notebook to Pipeline.....                        | 21 |
| Why pipelines? .....                                          | 21 |
| Step 1: Explore the Orchestration tree.....                   | 21 |
| Step 2: Review the DAG detail panel .....                     | 22 |
| Step 3: Trigger a DAG run.....                                | 22 |
| Step 4: Paused DAG handling .....                             | 23 |
| Step 5: Monitor the run.....                                  | 23 |
| Step 6: Inspect task logs .....                               | 23 |
| Step 7: Trace the MLflow lineage.....                         | 24 |
| Step 8: The promotion workflow .....                          | 24 |
| Where to go next.....                                         | 25 |
| Page 5: Data Quality .....                                    | 26 |
| What is Evidently? .....                                      | 26 |
| Step 1: Read the Data Health dot .....                        | 26 |
| Step 2: Understand which reports the pipeline generates ..... | 27 |
| evidently_quality .....                                       | 27 |
| evidently_drift.....                                          | 27 |
| Step 3: Open Evidently UI .....                               | 27 |
| Step 4: Read a data quality report .....                      | 28 |
| Step 5: Read a drift report.....                              | 28 |
| Step 6: Track trends over time .....                          | 28 |
| Where to go next.....                                         | 29 |
| Page 6: Serving & Deploying Models .....                      | 30 |
| What is noted-serving? .....                                  | 30 |
| Step 1: Register a model from a run .....                     | 30 |
| Step 2: Open the Model Registry view.....                     | 31 |
| Step 3: Deploy a version.....                                 | 31 |
| Step 4: Test the deployed model with Try It .....             | 31 |
| Step 5: Unload the current model .....                        | 32 |
| Step 6: Inspect the Logged Model artifacts .....              | 32 |
| Step 7: Consuming the model from an external client .....     | 33 |
| Where to go next.....                                         | 33 |
| Page 7: noted Assistant .....                                 | 34 |
| What is the noted Assistant? .....                            | 34 |
| Choosing a model .....                                        | 34 |
| Local models (via llama-cpp-python).....                      | 34 |
| Cloud models (via Anthropic).....                             | 35 |

|                                               |    |
|-----------------------------------------------|----|
| Skills .....                                  | 35 |
| MCP tools .....                               | 36 |
| Example 1: Explain a training run .....       | 36 |
| Example 2: Compare two runs .....             | 36 |
| Example 3: Debug a failed Airflow task .....  | 37 |
| Example 4: Hydra configuration question ..... | 37 |
| The “right panel” lifecycle .....             | 37 |
| Where to go next.....                         | 37 |

## Page 1: Your First Project

**Goal:** Create a new project, set up a Python environment, open a notebook, and run your first cell. By the end of this page you will have a working notebook connected to a Python kernel with your installed packages ready to use.

**Time:** ~5 minutes

### The noted interface

When you open noted in your browser, you see a layout similar to a modern code editor:

- A narrow **icon bar** on the left (vertical strip of icons).
- A **sidebar panel** (Explorer) that opens when you click an icon.
- A **center pane** where notebooks and files open.
- A **right panel** that hosts the noted Assistant (collapsible).

The Explorer is organized into top-level sections that each represent a capability of noted: **Projects**, **Experiments**, **Data** (Catalog and Storage), **Orchestration**, **Models** (Registry and APIs), **Environments**, **Assistant**, and **Knowledge Base** (Graph plus document categories).

Most operations in noted start with a click on one of these sections.

### Step 1: Create a project

1. Click the **folder icon** on the icon bar to open the Explorer if it is not already visible.
2. Double-click **Projects** to expand it, then single-click **Projects** to select it.
3. The Explorer title bar shows a green **Create Project (+)** button. Click it.
4. Enter a project name (for example my-first-project) and confirm.

Your new project appears under Projects in the tree.

**Tip:** noted follows a “select first, then act” pattern. Most actions appear in the title bar and change depending on the node you have selected in the Explorer tree. You can also right-click any node to see a context menu with the same actions.

### Step 2: Create a notebook

1. Double-click your new project to expand it.
2. Right-click the project name and choose **New Notebook**.
3. Name the notebook (for example experiment.ipynb) and confirm.

The notebook opens in the center pane with one empty code cell.

## Step 3: Create a Python environment

Before you can run code, you need a Python environment. noted manages Python, JavaScript, and R environments under the **Environments** section.

1. In the Explorer, double-click **Environments** to expand it. You see language sub-nodes: **Python**, **JavaScript**, **R**.
2. Double-click **Python** to expand it. You see the Python versions available on the host (typically 3.10 through 3.14).
3. Right-click on a version (for example Python 3.12) and choose **Create Environment**.
4. Enter an environment name (for example my-env) and confirm.
5. noted creates the virtual environment in a few seconds and a toast notification confirms success.

The new environment appears as a child of its Python version under the Environments section.

## Step 4: Install packages

A fresh environment contains only pip and setuptools. To install packages you need for your work:

1. In the Explorer, click your environment. The center pane shows the environment detail page with a package list and an install control.
2. Type package names into the install input (for example numpy pandas matplotlib) and click **Install**.
3. A terminal opens showing live pip output. When the install finishes, the package list refreshes.

You can install any packages available on PyPI the same way, either one at a time or several at once.

## Step 5: Connect the environment to your notebook

The notebook needs to know which environment to use for running cells.

1. Click the notebook's tab in the center pane to focus it.
2. In the notebook's **top bar**, click the **kernel selector**. It shows the current kernel status (initially **No kernel**).
3. A list of available environments appears. Pick the environment you just created (for example my-env (Python 3.12)).
4. The kernel status indicator turns green when the kernel is ready (~5 seconds).

The notebook is now bound to your environment for the lifetime of the session.

## Step 6: Run your first cell

1. Click inside the empty code cell.
2. Type:

```
import numpy as np
print(f"numpy version: {np.__version__}")
print("Hello from noted!")
```

3. Press **Shift+Enter** to run the cell (or click the play button in the cell toolbar).
4. The output appears below the cell.

You now have a working notebook.

## Where to go next

- **Page 2 - Configuring an Experiment** shows how to use noted's Hydra Composer to parameterize your training code so you can sweep across many configurations without editing the notebook.
- **Page 3 - Running an Experiment** explains how the Run Manager wraps your notebook execution in an MLflow run that captures parameters, metrics, data lineage, and the full Hydra configuration bundle for reproducibility.
- **Page 7 - noted Assistant** introduces the in-product AI assistant that knows all of noted's capabilities and can answer questions or perform tasks for you.

## Page 2: Configuring Your Experiment

**Goal:** Set up a Hydra configuration for your project, use the Configuration Composer to pick options and overrides, and use the Time Machine to reproduce or iterate from any past MLflow run.

**Prerequisite:** A project with a notebook and a Python environment (Page 1).

**Time:** ~10 minutes

### What is Hydra configuration?

Hydra is a configuration framework that organizes experiment settings as YAML files grouped by concern: model, data, training, scaler, and so on. Instead of hardcoding values like `epochs = 50` in a notebook, you define them in a config file and noted injects the resolved configuration into the running kernel automatically as a variable called `cfg`.

In Hydra’s mental model, the files under `config/` are **templates**, not “the config”. A run is defined by:

- which option you picked from each group, and
- any per-parameter overrides.

noted follows this model literally. Your notebook’s metadata records the selections and overrides. At run time, noted composes them with the baseline YAML files and injects the result into the kernel.

This gives you:

- Hyperparameter changes without editing code.
- Side-by-side comparison of experiments that used different configs.
- Full lineage of which config produced which run.
- One-click reproduction or iteration from any past MLflow run.

### Step 1: Create the config directory structure

Hydra configs live in a `config/` folder at the root of your project. Each subfolder is a “config group” (for example `model`, `data`, `training`).

A typical layout looks like:

```
config/
  config.yaml          (main entry point)
  model/
    gru_baseline.yaml
  data/
    default.yaml
```

```
training/  
  default.yaml
```

Create each folder and file using right-click > **New Folder** / **New File** on the project or its subfolders.

## Step 2: Write the config files

Click each YAML file to open it in the editor.

**config/config.yaml** (main entry point):

```
defaults:  
  - model: gru_baseline  
  - data: default  
  - training: default  
  
seed: 42
```

**config/model/gru\_baseline.yaml:**

```
type: GRU  
units: 128  
dropout: 0.2
```

**config/data/default.yaml:**

```
file: data/my_dataset.csv  
features:  
  - temperature  
  - humidity  
  - pressure  
target: temperature  
lookback: 120  
horizon: 24  
split:  
  train: 0.7  
  val: 0.15  
  test: 0.15
```

**config/training/default.yaml:**

```
epochs: 50  
batch_size: 128  
learning_rate: 0.0002
```

Save each file with **Ctrl+S**.

### Step 3: Enable Hydra view on the config folder

Once the config files are saved, enable the Hydra-aware view on the config folder so noted understands the structure.

1. Expand your project in the Explorer to reveal the config/ folder.
2. Right-click config/ and choose **Enable Hydra View**.
3. The folder icon changes to the Hydra icon to confirm Hydra view is active.

With Hydra view enabled:

- Clicking the config/ folder shows the **Configuration overview**: the config directory, all groups with option counts and defaults, discovered parameters, and a **Compose Config** button.
- Clicking a group subfolder (for example config/model/) shows the **group detail**: available options and which one is the default.
- Clicking individual YAML files still opens them in the editor for manual editing.

The Hydra view setting is stored per-project in .noted/settings.json and persists across sessions. To revert, right-click the config folder and choose **Disable Hydra View** - it returns to a normal directory view.

### Step 4: Open the Configuration Composer

The Configuration Composer is the panel where you pick group options, adjust overrides, and preview the resolved YAML.

1. Open a notebook in your project.
2. In the notebook's top bar, click the **Hydra icon** button.
3. The Composer opens.

You can also open it from the config folder's detail view by clicking the **Compose Config** button there.

The Composer has a top header with two modes and a two-column body.

**Top header:**

- **Baseline label** - what the Composer is reading from:
  - Baseline: Local when reading from the project's config/ folder.
  - Baseline: Run xxxxxx when reading from a past MLflow run's archived config bundle.
- **Mode toggle** - two buttons, Local Baseline and Experiment Run. Toggling is **preview-only**: the notebook is not changed until you click Apply.

- **Experiment and Run dropdowns** - enabled in Experiment Run mode. List experiments and runs for the current project that have a Hydra bundle archived.
- **Apply to Notebook button** - writes the current state (selections + overrides + baseline source) into the notebook's metadata. Nothing persists until this is clicked. In Experiment Run mode the button is disabled until an actual run is selected, so you cannot accidentally clear the notebook's selections.

**Left column:**

- **Config Groups** - one dropdown per group, pre-populated with the current selection.
- **Overrides** - one input per parameter, pre-populated with the current value. Changing an input to a different value records it as an override.
- **Templates** - save, load, or delete named combinations of selections and overrides (Local mode only).
- **Compose** and **Copy YAML** buttons.
- **Source Files** - which YAML file contributed each top-level key in the resolved config.
- **Hash** - the SHA-256 of the resolved config, used as a lineage tag on MLflow runs.

**Right column:**

- **Resolved Config** - the full composed YAML with syntax highlighting.

Changing a dropdown or an override updates the resolved preview instantly.

## Step 5: Apply selections to the notebook

The Composer is a preview. Nothing is written to the notebook until you click **Apply to Notebook** at the top right.

On Apply, the Composer writes three fields into notebook.metadata.noted:

```
{
  "notebook_uid": "<uuid-v4>",
  "hydra_baseline_source": "project://config/",
  "hydra_selections": {
    "group_selections": { "model": "gru_baseline", "data": "default" },
    "overrides": { "training.epochs": "10" }
  }
}
```

notebook\_uid is generated automatically the first time a notebook uses Hydra, and is stable across renames.

From this point on, every cell run in this notebook receives a freshly composed cfg variable built from the recorded selections and overrides.

## Step 6: The baseline badge in the notebook bar

Next to the Hydra icon in the notebook's top bar, noted shows a badge with two parts: a **label** and a colored **dot**.

**Label** - what the notebook is reading from:

- **BASELINE** - the local config/ folder (the normal case).
- **RUN xxxxxx** - pinned to an archived MLflow run's baseline (six characters of the run ID). Clicking the badge jumps to that run in the Explorer tree.

**Dot** - whether the current selections match expectations:

- **Green check** - everything is consistent. For a local baseline, this means the schema defaults are in use. For a pinned run, the current selections exactly match the archived run's selections.json.
- **Orange exclamation** - drift detected. For a local baseline, you have custom selections or overrides on top of the defaults. For a pinned run, your current selections diverge from the archived run's snapshot. Hovering the badge shows a tooltip listing exactly which keys differ.
- **Red X** - the baseline is unreachable. The archived MLflow run is missing, the bundle is incomplete, or MLflow is unavailable. The tooltip names the failure. Open the Composer to pick a different baseline.

The badge updates live when you Apply changes in the Composer.

## Step 7: Use the config in your notebook

Every cell execution automatically injects the resolved configuration as a variable named `cfg`.

Create a new code cell and type:

```
print(type(cfg))
print(cfg.model.type)
print(cfg.training.epochs)
print(cfg.data.features)
```

Run the cell. You should see:

```
<class 'omegaconf.dictconfig.DictConfig'>
GRU
50
['temperature', 'humidity', 'pressure']
```

The `cfg` object is an OmegaConf DictConfig - you can access nested values with dot notation (`cfg.model.type`) or dict notation (`cfg['model']['type']`).

**Note:** OmegaConf must be installed in your environment for dot notation. Install it via the Environments panel if it is not already present.

## Step 8: Per-run Hydra bundle logging

Every time a cell executes with a Hydra config active, noted logs a self-contained hydra/ folder to the MLflow run:

```
run artifacts:
  hydra/
    config/                (full baseline YAML tree)
      config.yaml
      model/gru_baseline.yaml
      data/default.yaml
      training/default.yaml
    selections.json        (group_selections + overrides used)
    resolved.yaml          (the composed output)
```

This bundle is **self-contained**. Each run captures the exact configuration that was injected into its kernel, regardless of whether the baseline came from the local `config/` folder or from a past run. No cross-run dependency chains.

The SHA-256 hash of `resolved.yaml` is also tagged on the MLflow run as `noted.hydra_config_hash`, usable as a lineage label.

## Step 9: Travel back in time

The Composer's Experiment Run mode lets you load any past run's Hydra baseline and use it for the current notebook.

1. Open the Configuration Composer from the notebook bar.
2. Click **Experiment Run** in the top header.
3. In the **Experiment** dropdown, pick the experiment that contains the run you want to reproduce.
4. In the **Run** dropdown, pick the run. Only runs with archived Hydra bundles appear in the list.
5. The Composer replaces its schema with the archived version and pre-populates selections and overrides from the archived `selections.json`.
6. Review and optionally tweak selections or overrides.
7. Click **Apply to Notebook**.

The notebook's metadata is updated:

- `hydra_baseline_source` becomes `mlflow://<run_id>`.
- `hydra_selections` reflects your (possibly tweaked) selections.
- The notebook-bar badge changes to RUN <6 chars>.

From this point on, every cell execution in this notebook composes against the **archived** baseline files, not the current local config/. The archived files are fetched once and cached in memory. Failures (MLflow unreachable, run deleted, bundle missing) produce a clear error that names the pointer and the failure - noted never falls through to a different source.

When a cell runs under an Experiment Run baseline, a new MLflow run is created for it, and that new run logs its own fresh hydra/ bundle containing the same baseline files the notebook used. This preserves the self-containment guarantee: every run can be reproduced from its own bundle alone.

To switch back to local baseline, open the Composer again, click **Local Baseline**, and click Apply.

## Step 10: Creating new baselines from past runs

noted deliberately does not automate promoting a past run's config back into the local config/ folder. If you find a past run whose configuration you want to become the new team baseline:

1. Open the MLflow UI for that run.
2. Download the hydra/config/ folder from its artifacts.
3. Copy the files into your project's config/ folder.
4. Commit to Git if you want version history.

This is an explicit choice. noted never writes to config/ on your behalf.

## Step 11: Run your experiment with MLflow tracking

The **Run Manager** is the panel that wraps a notebook execution in an MLflow run. It handles dataset registration, run naming, and the MLflow lifecycle so you do not have to add tracking boilerplate to your cells.

### Opening the Run Manager:

1. In the notebook's top bar, click the **Experiments icon** (test tube, second icon group on the left).
2. A panel titled **Experiments** opens.

### Creating a run definition:

1. Click **Add Run** in the panel toolbar.
2. Give the run a name (for example baseline-gru).

3. In the notebook, cells now show a checkbox on hover. Check the cells you want to include in this run, or click **Select All** in the panel.

### Dataset tracking:

- If your notebook uses a Hydra config, the dataset section shows a single read-only row with the filename from `cfg.data.file`, a Hydra icon, and the label “from Hydra config”. noted derives the dataset automatically - switching datasets is done via the Composer, not here.
- If your notebook does not use Hydra, a DVC-tracked file picker appears so you can select which datasets this run uses.

### Executing:

1. Click the **play button** next to the run name.
2. noted wraps the selected cells in an `mlflow.start_run()` context, executes them in order, and registers the DVC dataset hashes with the MLflow run.
3. When complete, the new run appears in the Experiments section of the Explorer tree with full metrics, parameters, and the hydra/ bundle.

## Step 12: Change parameters and re-run

The power of Hydra is changing parameters without editing code:

1. Open the Configuration Composer.
2. Change `training.epochs` from 50 to 10 in the Overrides section.
3. The resolved YAML on the right updates automatically.
4. Click **Apply to Notebook**.
5. Execute the run via the Run Manager. `cfg.training.epochs` now returns 10, and the new MLflow run carries its own hydra/ bundle reflecting the override.

Compare the two runs in the Experiments section of the Explorer: select both, right-click, and choose **Compare Runs** for a side-by-side diff of metrics and parameters.

## Where to go next

- **Page 3 - Running an Experiment** goes deeper on the Run Manager, kernel sessions, live metrics streaming, and the Experiment Run browser.
- **Page 4 - From Notebook to Pipeline** shows how to promote a successful notebook run into an Airflow DAG for scheduled training.
- **Page 6 - Serving & Deploying Models** explains how to deploy a registered model into the serving endpoint and test it with the Try It panel or an external client.

## Page 3: Running Your Experiment

**Goal:** Execute a training run with MLflow tracking, monitor it live, explore the results in the Explorer, and compare it against other runs.

**Prerequisite:** A project with a Hydra config and a notebook that uses cfg (Pages 1 and 2).

**Time:** ~15 minutes, plus the time it takes to train.

### What is MLflow?

MLflow is the tracking backend that records what happened during each training run: the parameters you used, the metrics that resulted, and the artifacts (models, plots, config bundles) that were saved.

In noted, every run created via the Run Manager is automatically registered in MLflow. You do not write any MLflow boilerplate. noted handles `mlflow.start_run()`, parameter logging, and artifact storage transparently.

Runs are grouped into **experiments**. noted uses your **project name** as the experiment name, so all runs from a project land in the same experiment automatically.

### Step 1: Define a run in the Run Manager

If you completed Page 2 Step 11, you already have a run definition. If not:

1. Open the notebook.
2. Click the **Experiments icon** (test tube) in the notebook toolbar.
3. Click **Add Run** and give it a name (for example baseline-gru).
4. Tag the cells you want to include by hovering over each cell and checking the checkbox, or click **Select All** in the Run Manager.

The dataset row fills in automatically:

- If the notebook uses a Hydra config, a read-only row labeled **“from Hydra config”** shows the filename from `cfg.data.file`. No selection needed; it derives from what the Composer set.
- If no Hydra config is in use, a DVC-tracked file picker appears with checkboxes.

Run definitions (name and cell membership) are stored in the notebook’s metadata, so they persist across sessions.

### Step 2: Execute the run

1. In the Run Manager panel, click the **play button (▶)** next to your run name.
2. noted:
  - Resolves the dataset DVC hash (from the Hydra config or your manual picks).

- Sets the MLflow experiment to your project name.
  - Opens an MLflow run with the name you defined.
  - Executes the tagged cells in order inside that run context.
3. Cell outputs appear in the notebook as cells execute.
  4. Live metrics stream to the Metrics panel as training progresses (Step 3).

When execution finishes, the run closes and appears in the **Experiments** section of the Explorer tree.

**Important:** only cells executed via the Run Manager are wrapped in an MLflow run. Running cells manually via the notebook's **Run All** button or the Shift+Enter cell-by-cell flow does **not** open an MLflow run. Cells will execute, but no parameters, metrics, model, or lineage will be captured. Always use the Run Manager play button for tracked experiments.

### Step 3: Monitor live metrics during training

If your training code calls `mlflow.log_metric()` (or uses noted's `_MLflowEpochLogger` Keras callback), metrics stream to the **Metrics panel** in real time.

The Metrics panel opens automatically when a run starts. You can also open it manually from the notebook toolbar.

#### Panel views:

- **Split** (default): one chart per metric, two columns side by side.
- **Combined:** all metrics overlaid on a single chart.
- **Table:** summary table of final metric values.

**Epoch progress bar:** when epoch-like metrics are detected, a progress bar appears showing current epoch / total epochs with a percentage, useful for estimating remaining time on long training runs.

**Historical data:** you can also open the Metrics panel for a past run from its detail page in the Explorer. Each run opens an independent panel, so you can monitor two runs side by side.

### Step 4: Explore results in the Experiments tree

Once a run finishes, click **Experiments** in the Explorer icon bar to see the tree.

```
Experiments
└─ your-project-name          (experiment)
   │ ✓ 2026-04-13 - baseline-gru    (finished run)
   │ ► 2026-04-13 - fast-experiment (running run)
   └─ ✗ 2026-04-12 - broken-run     (failed run)
```

Run status icons:

- **Green check** - FINISHED
- **Blue play** - RUNNING
- **Red X** - FAILED
- **Orange stop** - KILLED

Clicking a run opens its **detail panel**, which shows:

- **Status, Run ID** (full ID in monospace), **Started / Ended / Duration**.
- **Metrics** - final metric values in a sorted key-value grid.
- **Parameters** - all logged params in a sorted grid.
- **Tags** - run tags (MLflow system tags hidden by default).
- **Inline charts** - one line chart per metric with 2+ data points, two columns, interactive (step on X axis, value on Y axis).
- **Action buttons** (for finished runs):
  - **Register Model** - promotes the run's model to the Model Registry.
  - **Create Snapshot** - captures a point-in-time snapshot of the run.
  - **Ask Assistant** - opens the AI assistant pre-loaded with the run's context (Step 6).

**Artifacts:** scroll down in the detail panel to see the run's stored artifacts, organized into categories:

- **Models** - saved model files.
- **Images** - plots and figures.
- **HTML Charts** - interactive reports (Evidently, etc.).
- **Files** - everything else, including the `hydra/` bundle (`hydra/config/`, `hydra/selections.json`, `hydra/resolved.yaml`).
- **Logged Models** - the MLflow 3.x Logged Model entities produced by `log_model()` calls. Each Logged Model contains the standard `MLmodel`, `conda.yaml`, `python_env.yaml`, `requirements.txt`, and the framework-specific weights under `data/`. Clicking any file in the tree previews it inline; `requirements.txt` and the YAML files open with syntax highlighting.

## Step 5: The Lineage view

Click a **model version** in the Registry (Models > Model Registry) to see its full lineage chain:

Data (DVC) → Config (Hydra) → Pipeline (Airflow) → Code (Git) → Run (MLflow) → Model (Registry)

Each layer is a card showing the relevant identifier:

- **Data (DVC)** - the dataset file path and its DVC hash.
- **Config (Hydra)** - the `noted.hydra_config_hash` for that run.
- **Pipeline (Airflow)** - the DAG ID and DAG run ID (if the run was produced by an Airflow pipeline).
- **Code (Git)** - the Git commit SHA and branch of the project at training time.
- **Run (MLflow)** - the MLflow run ID, name, and status. Clicking jumps to the run in the Explorer tree.
- **Model (Registry)** - the model name, version, and any aliases.

Missing layers show Not tracked in grey. A complete chain means every component needed to reproduce the model is identified and addressable.

## Step 6: Compare two runs

Comparing runs lets you see side-by-side diffs of metrics and parameters to understand what changed.

1. In the Experiments tree, click the first run to open its detail.
2. In the detail panel, click **Compare**.
3. A picker appears listing all other runs in the same experiment.
4. Select the second run.

A comparison panel opens showing:

- **Side-by-side headers** with run names and IDs (color-coded).
- **Metrics diff table** - both values and the numeric delta (with percent change) between runs; rows with differences are highlighted.
- **Parameters diff table** - side-by-side params for both runs.
- **Tags diff table** - non-system tags for both runs.
- **Overlaid metric charts** - two-column grid of time-series charts with both runs drawn on the same axes for direct visual comparison.
- **Explain Differences** button - sends both run IDs and names to the Assistant (Step 7).

## Step 7: Ask the Assistant about a run

Every run detail panel has an **Ask Assistant** button. Clicking it:

1. Opens (or focuses) the Assistant panel on the right.
2. Pre-populates the chat with a message that includes the full run ID and run name, asking the Assistant to analyze its metrics and parameters.

The comparison panel has an **Explain Differences** button that works similarly, sending both run IDs and names to the Assistant.

The Assistant can retrieve additional context via MCP tools (`get_notebook_cells`, `mlflow-run-interpretation`, and others) to provide deeper analysis. Page 7 covers the Assistant in full.

## Step 8: Reproduce or iterate from a past run

If a run performed well and you want to reproduce it exactly or iterate from it, use the **Time Machine** (covered in Page 2, Steps 9-10):

1. Find the run in the Experiments tree.
2. Open the Configuration Composer from the notebook bar.
3. Switch to **Experiment Run** mode.
4. Select the experiment and the run from the dropdowns.
5. The Composer loads the archived hydra/ bundle from that run and pre-populates selections and overrides.
6. Optionally tweak overrides, then click **Apply to Notebook**.
7. Execute a new run via the Run Manager.

The new run carries its own fresh hydra/ bundle, keeping the self-containment guarantee for all runs.

## Where to go next

- **Page 4 - From Notebook to Pipeline** shows how to promote a successful notebook run into an Airflow DAG for scheduled training.
- **Page 5 - Data Quality** introduces DVC for dataset versioning and Evidently for drift detection.
- **Page 6 - Serving & Deploying Models** explains how to deploy a registered model into the serving endpoint and test it.

## Page 4: From Notebook to Pipeline

**Goal:** Trigger an Airflow DAG from noted, monitor its execution, debug failures, and trace the resulting MLflow run back to its source.

**Prerequisite:** A project with a working notebook experiment (Pages 1-3). A DAG that mirrors the notebook's training logic must already exist in the Airflow dags/ folder.

**Time:** ~10 minutes, plus pipeline execution time.

### Why pipelines?

A notebook experiment is interactive and exploratory. A pipeline is repeatable and scheduled. Once you have validated an approach in the notebook, you promote it to an Airflow DAG so it can:

- Run on a schedule (nightly retraining, fresh data ingestion).
- Run on larger infrastructure without occupying your browser.
- Be triggered by upstream events (new data landed in storage).
- Produce MLflow runs with identical config tracking as your notebook.

In noted, DAG runs and notebook Run Manager runs land in the **same MLflow experiment**. There is no gap: a model trained by the DAG is visible in the same Experiments tree as one trained interactively.

### Step 1: Explore the Orchestration tree

Click the **Orchestration icon** in the Explorer icon bar (network diagram icon) to open the Orchestration section.

```
Orchestration
└─ jena_training_pipeline      (DAG, active)
    │ 2026-04-13 11:00 - success (run)
    │   │ ingest_data          (task)
    │   │ preprocess_data      (task)
    │   │ train_model_task      (task)
    │   │ log_hydra_lineage     (task)
    │   │ promote_model         (task)
    │   └─ evidently_drift      (task)
    └─ 2026-04-12 23:00 - failed (run)
```

DAG status icons: play = active, pause = paused. Run status icons: green = success, red = failed, blue = running, orange = queued.

Clicking a **DAG node** opens the DAG detail panel. Clicking a **run node** opens the run detail panel. Clicking a **task node** opens the task log viewer.

---

## Step 2: Review the DAG detail panel

Click the DAG name (for example `jena_training_pipeline`) to open its detail panel. You see:

- **Status badge** - Enabled or Paused.
- **Description** - from the DAG's `doc_md` or `description` field.
- **Schedule** - cron expression or None.
- **Task graph** - SVG dependency diagram of all tasks in the DAG.
- **Schedule editor** - change or clear the cron schedule directly from noted, with presets (@hourly, @daily, @weekly, "Every 6h").
- **Pause / Unpause** button - toggle the DAG's active state.
- **Run DAG** button - opens the trigger panel (Step 3).
- **Run history table** - last 10 runs with started time, duration, state, and **lineage chips** (Step 7).

## Step 3: Trigger a DAG run

Click **Run DAG** in the DAG detail panel. A trigger panel opens titled `Run DAG: {dag_id}`.

If your project uses **Hydra config**, the trigger panel shows a **Hydra Configuration** section at the top:

- One dropdown per config group (data, model, scaler, and so on).
- Changing a dropdown recomposes the config and auto-fills the corresponding param fields below.
- A **Hydra config hash** is displayed and will be tagged on the resulting MLflow run.

**DAG Parameters section:** below the Hydra section (or at the top if no Hydra config), each DAG parameter appears as a typed input field:

- Text or number fields with descriptions and default values.
- Checkboxes for boolean parameters.
- Dropdowns where the DAG defines a fixed set of choices.

**Useful buttons in the trigger panel:**

- **Load Last Run Config** - pre-fills all fields from the most recent successful run. Essential for "run again with one change" workflows.
- **Custom JSON** - text area to merge arbitrary extra fields into the run conf, for edge cases not covered by the param schema.

**Triggering:** click **Trigger** at the bottom of the panel. On success, the panel shows the new run ID and initial state (usually queued). The run appears in the Orchestration tree within a few seconds.

## Step 4: Paused DAG handling

If you try to trigger a paused DAG, a modal appears with three options:

- **Cancel** - abort the trigger.
- **Keep Paused & Queue Run** - submit the run but leave the DAG paused; the run sits in the queue and does not execute until the DAG is manually unpaused.
- **Unpause & Run Immediately** - unpauses the DAG and triggers the run in one action (the most common choice).

You can also toggle the DAG's paused state directly from the DAG detail panel using the **Pause / Unpause** button.

## Step 5: Monitor the run

Click the new run node in the Orchestration tree to open its detail panel.

The run detail shows:

- **State badge** - queued → running → success / failed.
- **Run ID** - the full dag\_run\_id.
- **Timestamps** - logical date, actual start, end.
- **Config** - the full JSON config passed to the run (formatted).
- **Task graph** - SVG of task dependencies, nodes colored by current task state (green = success, red = failed, blue = running, grey = pending).
- **Task list** - sortable rows showing start time, state icon, task ID, operator type, and duration. Click any row to open the task log.

**Stop Run** button: if the run is in running or queued state, a red **Stop Run** button appears at the top of the detail panel. Clicking it marks the run as failed and stops execution.

Re-click the run node to get the latest state.

## Step 6: Inspect task logs

Click any task row in the run detail to open the **task log viewer**. It shows a full xterm.js terminal with the task's stdout/stderr output (ANSI colors rendered, 5000-line scrollbar).

- If the task is running or queued, logs auto-refresh every 3 seconds.
- **Copy Log** button copies the full log text to the clipboard.

- **Retry Task** button (for failed tasks) clears the task state so Airflow re-queues it on the next scheduler tick. Use this after fixing the underlying issue.

The task log viewer has an **Ask Assistant** button. Clicking it sends the last 1000 characters of the log plus the task state and names to the AI assistant, pre-loaded with context for error analysis - useful for quickly diagnosing a failed task without leaving noted.

## Step 7: Trace the MLflow lineage

After a DAG run completes, the **Run History table** in the DAG detail panel shows a **Lineage** column with clickable chips:

- **MLflow chip** (blue, flask icon) - shows the first 10 characters of the MLflow run ID. Clicking it expands the Experiments section of the Explorer tree, finds the matching run, and navigates to it, switching context from Orchestration to Experiments in one click.
- **Hydra hash chip** - the SHA-256 of the resolved config, matching the `noted.hydra_config_hash` tag on the MLflow run.
- **DVC hash chip(s)** - dataset hashes registered with the run.

The MLflow run produced by the DAG lands in the **same experiment** (project name) as all Run Manager runs from that project. It has:

- Full metrics and parameters logged by the training task.
- A hydra/ artifact bundle uploaded by the `log_hydra_lineage` task: `hydra/config/` (full YAML tree), `hydra/selections.json`, `hydra/resolved.yaml`.
- The `noted.hydra_config_hash` tag.

This means the DAG-produced run is fully compatible with the Time Machine: you can load it in the Composer's Experiment Run mode, inspect its config, and reproduce or iterate from it exactly like any Run Manager run.

## Step 8: The promotion workflow

A typical promotion workflow from notebook to pipeline looks like this:

1. **Notebook** (interactive): run experiments via Run Manager, compare results, identify the winning config.
2. **Time Machine**: load the winning run in the Composer to capture its exact config.
3. **DAG**: ensure the DAG's parameters match the Hydra groups and override keys from the Composer.
4. **Trigger**: use the Hydra dropdowns in the trigger panel to select the same config as the winning run.

5. **Verify:** after the DAG run completes, use the MLflow chip to navigate to the DAG's MLflow run and compare it against the notebook run. Metrics should match within noise.

## Where to go next

- **Page 5 - Data Quality** covers DVC for dataset versioning and Evidently for drift detection - the inputs to any reliable pipeline.
- **Page 6 - Serving & Deploying Models** explains how to deploy the model produced by this pipeline into the serving endpoint.

## Page 5: Data Quality

**Goal:** Understand how noted uses Evidently to track data quality and feature drift, read the Data Health indicator in the Explorer, and explore detailed reports in the Evidently UI.

**Prerequisite:** A project with a working Airflow pipeline (Page 4). The pipeline must include the `evidently_quality` and `evidently_drift` tasks.

**Time:** ~10 minutes.

### What is Evidently?

Evidently is an open-source library and UI service that computes statistical reports on datasets: summary statistics, missing values, distribution shapes, and feature-level drift scores. It is the monitoring engine behind noted's data quality surface.

noted's integration is intentionally thin. noted does not render its own drift charts or quality report panels. Instead, it:

- Surfaces a **Data Health dot** (green / yellow / red) on the Data node in the Explorer tree, sourced from the Evidently API.
- Provides a **full Evidently UI** as a service tab for deep exploration.
- Lets DAG tasks generate and save Evidently snapshots automatically.

The detailed dashboards, time-series trend panels, and per-feature drill-downs live in the Evidently UI. This is the same pattern as MLflow (noted shows run badges; MLflow shows detailed charts) and Airflow (noted shows task status; Airflow shows the Gantt view).

### Step 1: Read the Data Health dot

In the Explorer sidebar, click the **Data** icon. Look at the "Data" root node title - you may see a small colored dot to the right of the label:

- **Green dot** - the latest Evidently data quality report passed. Feature statistics look healthy (no missing values above threshold, distributions within expected ranges).
- **Yellow dot** - warning. One or more quality checks found potential issues (some missing values, a feature with unusual variance).
- **Red dot** - problem detected. At least one quality check failed (missing data above threshold, unexpected distribution, constant column).

Hovering the dot shows a short summary from the report.

The dot refreshes automatically when the Explorer loads. If no Evidently project exists yet for this noted instance, the dot is absent.

## Step 2: Understand which reports the pipeline generates

The jena\_weather training DAG (jena\_training\_pipeline) contains two Evidently tasks that run automatically as part of every pipeline execution.

### evidently\_quality

- **What it computes:** a DataSummaryPreset report on the engineered feature dataset - the full dataset after resampling and feature engineering, before the train/val/test split.
- **What it measures:** per-column statistics (mean, std, min, max, missing value count, value range, distribution shape) for all numerical feature columns.
- **Tags:** data-quality, jena-weather, pipeline.
- **When it runs:** after preprocess\_data completes, in parallel with train\_model\_task.

### evidently\_drift

- **What it compares:** training split (reference) vs test split (current). Detects whether the test distribution has drifted from the distribution the model was trained on.
- **What it measures:** a DataDriftPreset report - per-feature drift scores using statistical tests (KS test for numerical features), an overall dataset drift score, and per-feature drift labels (drifted / not drifted).
- **Tags:** drift, jena-weather, pipeline.
- **Metadata:** the run\_id of the corresponding MLflow training run, linking the drift report to the model trained on this split.
- **When it runs:** after train\_model\_task completes, in parallel with log\_hydra\_lineage and promote\_model.

Both tasks save their reports as **snapshots** to the Evidently workspace, where they accumulate over time and form the basis of trend dashboards.

## Step 3: Open Evidently UI

Click the **Evidently icon** in the bottom section of the icon bar. The Evidently UI opens as a tab in the center pane.

The Evidently UI is organized into **projects**. The jena\_weather pipeline saves all its reports to a project named Jena Weather (created automatically on first run if it does not exist).

Click the Jena Weather project. You see a dashboard with:

- **Snapshot list** - all saved reports, sorted by time, with tags and run metadata.
- **Panels** - configurable time-series charts tracking how metric values evolve across snapshots (drift score over successive pipeline runs, missing value rate over time, and so on).

---

## Step 4: Read a data quality report

In the Evidently project, filter by tag data-quality to see quality snapshots. Click any snapshot to open its interactive report.

A DataSummaryPreset report shows:

- **Dataset summary** - row count, column count, missing value rate, duplicate row count.
- **Per-column statistics** - for each numerical feature, a row with mean, std, min, max, missing %, and a small distribution histogram.
- **Column type summary** - confirms which columns are treated as numerical.

Use this to verify that the ingested data looks as expected before trusting training results. If the temperature column shows an unexpected mean or a spike in missing values, the model trained on that data is unreliable regardless of its loss metrics.

## Step 5: Read a drift report

Filter by tag drift to see drift snapshots. Click any snapshot to open the drift report.

A DataDriftPreset report shows:

- **Dataset-level drift verdict:** “Dataset Drift Detected” or “No Drift Detected” based on the share of drifted features.
- **Per-feature drift table:** one row per feature, with the drift score, the statistical test used, and a label.
- **Distribution plots:** for each feature, overlaid histograms of the reference (training) and current (test) distributions.

For the jena\_weather dataset, the training split is early years and the test split is the most recent period. Some drift is expected and acceptable. Significant drift in the temperature or pressure features may indicate a seasonal shift that the model has not seen.

The run\_id metadata on each drift snapshot links back to the MLflow training run. To trace a drift finding to the model it affects:

1. Copy the run\_id value from the Evidently report metadata.
2. Switch to the Experiments section in the Explorer.
3. Search for the run with that ID to see its metrics and parameters.

## Step 6: Track trends over time

The Evidently UI’s dashboard panels show how quality and drift metrics evolve across pipeline runs. After several pipeline executions, you can:

- See whether data quality is improving or degrading.
- Detect seasonal drift patterns in the weather features.
- Correlate drift score increases with drops in model performance by cross-referencing the MLflow run IDs in the snapshot metadata.

To add a custom panel, use Evidently's panel builder in the project dashboard. For example, add a "Drift Share" time-series panel that tracks the percentage of drifted features per pipeline run.

## Where to go next

- **Page 6 - Serving & Deploying Models** explains how to deploy a model from the Registry into the serving endpoint and test it.
- **Page 7 - noted Assistant** introduces the in-product AI assistant that can interpret data quality reports, drift scores, and run lineage with the MCP tools built into noted.

## Page 6: Serving & Deploying Models

**Goal:** Register a trained model, deploy it into the serving endpoint, test it with the built-in Try It panel, and understand how external clients can consume it.

**Prerequisite:** A finished MLflow run that logged a model (Page 3 or Page 4).

**Time:** ~10 minutes.

### What is noted-serving?

noted-serving is a small FastAPI service that loads a registered MLflow model into memory and answers prediction requests over HTTP. It runs as its own container alongside the noted container and talks to the same MLflow tracking server.

Exactly one model is deployed at a time. Deploying a new model replaces the currently loaded one. This keeps the demo simple and the memory footprint small.

From noted's Explorer you can:

- **Register** a model version from a run.
- **Deploy** any registered version to noted-serving.
- **Unload** the currently deployed model.
- **Try It** by sending a synthetic input to the running model and seeing its prediction.
- Inspect the **Logged Model** artifacts (MLmodel, conda.yaml, python\_env.yaml, requirements.txt, and the framework-specific weights) that MLflow archived when the model was logged.

### Step 1: Register a model from a run

Open the **Experiments** section in the Explorer, drill down to the run that produced the model you want to serve, and click to open its detail panel.

Scroll to the action buttons and click **Register Model**.

A dialog asks for:

- **Model name** - the name the model will appear under in the Registry (for example Jena Weather Forecaster). Registering under an existing name creates a new version instead of a new model.
- **Aliases** (optional) - any aliases to attach to this version. A common pattern is to set champion on the version that downstream systems should pick up.

Click **Register**. The new version appears under **Models > Model Registry > {model name}** in the Explorer.

Each registered version has a lineage chain visible in its detail panel. See Page 3 Step 5 for the full explanation of the Data / Config / Pipeline / Code / Run / Model chain.

## Step 2: Open the Model Registry view

Click **Models > Model Registry** in the Explorer to see the full list of registered models and versions. For each version, the detail panel shows:

- **Version metadata** - version number, created timestamp, description, stage, aliases.
- **Lineage chain** - the five (or six) layer cards showing the data, config, pipeline, code, MLflow run, and the registered model itself.
- **Deploy / Unload / Try It** buttons - the three-button controller for serving.
- **Logged Models** section - the MLflow 3.x Logged Model entity linked to this run. Expanding it shows the archived artifact tree: MLmodel (with the model signature and flavor), conda.yaml, python\_env.yaml, requirements.txt, and the framework-specific data/ folder. Clicking any file opens a syntax-highlighted preview of its contents.

## Step 3: Deploy a version

Click **Deploy** on the version you want to serve.

noted opens a streaming progress card that shows the phases of the deploy as they happen:

- **resolving** - looking up the version in MLflow.
- **downloading** - fetching the model artifact locally.
- **loading\_model** - loading the weights into the serving process.
- **ready** - the model is live and ready to answer predictions.

A successful deploy takes a few seconds for a cached model and a few tens of seconds for a cold one (first time a given model is loaded after container start).

When the deploy completes, the state machine on the version card transitions to **Deployed here** and the **Unload** button becomes available. Other versions of the same model show **Deployed elsewhere** to make it clear where the currently-serving one lives.

If the deploy fails, the progress card shows an **error** phase with a clear message: MLflow unreachable, version not found, artifact download failed, or model load error. Noted never reports success if the load did not actually complete.

## Step 4: Test the deployed model with Try It

With a model deployed, click **Try It** on its version card. The Try It panel opens showing:

- **Current model** - name, version, and load time (confirms you are testing the right one).

- **Input form** - derived from the model's signature. For a tensor-input model, it shows the expected shape and dtype, with a **Generate Sample** button that builds a synthetic input matching the signature.
- **Predict** button - sends the input to the serving endpoint.
- **Prediction output** - rendered according to the model's output signature: a scalar, a vector plotted as a chart, or a JSON tree for structured outputs.

Try It is the fastest way to verify a deploy: deploy, click Try It, click Predict, see a result.

## Step 5: Unload the current model

Click **Unload** on the currently-deployed version. noted asks the serving container to drop the model and free its memory.

Unload is optional between deploys. Deploying a new version automatically unloads the previous one. Unload is useful when you want to:

- Free GPU memory without deploying anything new.
- Confirm a deploy / unload cycle as part of a smoke test.

After unload, all version cards for this model show **Not deployed**, and Try It is disabled until something is deployed again.

## Step 6: Inspect the Logged Model artifacts

The **Logged Models** section under the version card shows the MLflow 3.x Logged Model entity linked to this run. It is a separate artifact store from the run's own artifact tree - the model binary plus the environment files that describe how it was trained and what it needs at inference time.

Click any file to preview it inline:

- **MLmodel** - the YAML manifest with the model signature, flavor (tensorflow, pytorch, sklearn, and so on), loader module, and MLflow version. Syntax-highlighted.
- **requirements.txt** - the pip requirements that MLflow captured when the model was logged. This is what a production serving environment needs to install to run the model faithfully.
- **conda.yaml / python\_env.yaml** - conda and pip environment specs, again for serving-time reproducibility.
- **data/** - the framework-specific model files (for example model.keras for a Keras model, state\_dict.pt for PyTorch).

A **Download** button is available on each file so you can export any of them without leaving noted.

## Step 7: Consuming the model from an external client

The noted-serving container exposes a simple HTTP API that any external client can use:

- GET /health - current state (what model is loaded, load time, framework, parameter count).
- POST /load - streaming NDJSON response that loads a model by name and version (and optionally alias).
- POST /predict - sends an input to the currently loaded model and returns the prediction.
- GET /schema - the model's input and output signature, used by clients to build correct request payloads.

A working reference client ships with the platform at `iscte/jena_client/`. It is a small FastAPI + socket.io app that:

- Talks to MLflow directly via REST to list registered models and their versions and aliases.
- Presents three dropdowns (Model / Version / Alias) so a user can pick which version to load.
- Streams load progress from noted-serving's NDJSON endpoint and shows the phases in its UI.
- Builds a synthetic input matching the loaded model's schema, sends it to /predict, and renders the result as a chart plus a table.
- Applies the inverse scaler transform (using `target_mean` and `target_std` logged as MLflow params) to convert standardized model output back into real units when the model was trained on normalized data.

The `jena_client` is useful as a standalone demo of noted-serving and as a starting point for building a custom inference UI for your own models.

## Where to go next

- **Page 7 - noted Assistant** introduces the in-product AI assistant. It can answer questions about deployed models, run diagnostics, and trigger MCP tool calls such as model registration and run comparison.

## Page 7: noted Assistant

**Goal:** Understand what the noted Assistant can do, how to pick between local and cloud models, and how to use its skills and MCP tools to speed up common tasks.

**Time:** ~10 minutes.

### What is the noted Assistant?

The noted Assistant is an AI chat panel built into the platform. It is aware of noted's structure and can retrieve context, interpret it, and take actions on your behalf - without leaving the browser tab.

Concretely, the Assistant can:

- Explain an MLflow run's metrics and parameters in plain English.
- Compare two runs and summarize what changed and why it matters.
- Read a failed Airflow task log and point at the probable cause.
- Walk through the steps to set up a Hydra config for a new project.
- Use domain skills (Airflow, DVC, Evidently, Hydra, MLflow, noted) to answer "how do I...?" questions with accurate, platform-aware guidance.
- Invoke MCP tools such as listing a notebook's cells, fetching a run's artifacts, or retrieving the current Hydra selections - so it can act on live state rather than guess.

Open the Assistant by clicking the **speech bubble icon** in the icon bar, or by clicking **Ask Assistant** on any run, task log, or comparison panel.

### Choosing a model

The Assistant dropdown at the top of the chat panel lists the models available to this noted instance.

#### Local models (via llama-cpp-python)

noted ships with a local LLM served by llama-cpp-python. The default is **Gemma 4** (4B parameters, instruction-tuned), GPU-accelerated when CUDA is available. Local models:

- Run entirely inside the noted container - no API key required, no outbound calls, no data leaving your machine.
- Support native tool calling via Gemma 4's tool-call tokens, so the Assistant can invoke MCP tools just like the cloud models can.
- Are the right choice for experiments, sensitive data, offline demos, or when you want to stay cost-free.

## Cloud models (via Anthropic)

If `ANTHROPIC_API_KEY` is set in the environment, noted also exposes Claude models in the dropdown:

- **Claude Sonnet 4.6** - the default cloud model. Good balance of speed, cost, and reasoning quality for day-to-day use.
- **Claude Opus 4.6** - the most capable model for long or complex investigations (run triage, multi-step refactors, deep diagnostics).
- **Claude Haiku 4.5** - the fastest and cheapest option for short tasks and quick answers.

All Claude models have a 200k-token context window and support the full MCP tool surface.

Switching models mid-conversation is safe: the chat history is preserved and the next turn is answered by the newly-selected model.

## Skills

noted ships with approximately 40 **domain skills** that load automatically into the Assistant's context when relevant. Each skill is a short, curated document describing a specific capability or best practice of one of noted's integrated tools.

The skills cover seven domains:

**Airflow** - DAG creation, DAG overview, scheduling, performance, sweep strategies, task debugging, task dependencies, trigger configuration.

**DVC** - best practices, checkout, lineage, sync debugging, file tracking, versioning.

**Evidently** - data quality, drift detection, monitoring setup.

**Hydra** - composition, group structure, pipeline integration, initial setup, sweep design, template patterns.

**MLflow** - artifact management, hyperparameter analysis, model registration, reporting, run comparison, run debugging, run interpretation, serving, snapshots, training curve analysis.

**noted core** - auto-instrumentation, coding conventions, lineage, notebook resolution, platform overview, troubleshooting.

**General ML** - workflow guidance, Python linting, web fetch.

When you ask the Assistant a question, it picks the relevant skills for the topic and loads their content into context before generating an answer. You never need to select a skill manually; the routing is automatic.

## MCP tools

Beyond skills, the Assistant can call **MCP (Model Context Protocol) tools** to fetch live state or perform actions:

- `get_notebook_cells` - returns the source of every cell in the current notebook, so the Assistant can reason about actual code.
- `get_active_hydra_config` - returns the current Composer selections and the resolved config, so it can explain what will run.
- `mlflow-run-interpretation` - retrieves a run's full metrics, parameters, tags, and artifacts for analysis.
- `compare_runs` - retrieves two runs and produces a structured diff the Assistant can narrate.
- `airflow-task-debugging` - retrieves a task log and Airflow metadata for root-cause analysis.
- Tools for DVC lineage, registered model metadata, project files, and more.

Tool calls appear inline in the chat as collapsed "Tool call" blocks. You can expand them to see the exact arguments and the raw result the Assistant received before forming its answer.

### Example 1: Explain a training run

1. Open the Experiments tree and navigate to a finished run.
2. Click the **Ask Assistant** button at the top of the run detail panel.
3. The Assistant panel opens and is pre-populated with a message like: > Analyze run 92215c82... ("DEMO Run #3") in the jena\_weather > experiment. Explain its metrics and parameters and tell me if > the result looks reasonable.
4. Send the message. The Assistant calls `mlflow-run-interpretation` to fetch the run's full record, then produces a narrative covering the hyperparameters used, the final metric values, what the training curves look like, and any red flags (overfitting, exploding loss, missing metrics).

Useful for: triaging a large batch of runs, onboarding a team member to an unfamiliar experiment, generating a summary paragraph for a status update.

### Example 2: Compare two runs

1. Open a run detail and click **Compare**.
2. Pick a second run.
3. The comparison panel opens, showing the metrics / parameters / tags diff plus overlaid metric charts.

4. Click **Explain Differences**. The Assistant opens with both run IDs pre-loaded and analyzes what changed between them.
5. The Assistant calls `compare_runs` to get the structured diff, then explains which parameter changes caused which metric movements and whether the difference is statistically meaningful.

Useful for: understanding why a sweep's best run beat the baseline, verifying that an improvement is real and not noise, documenting a promotion decision.

### Example 3: Debug a failed Airflow task

1. Open the Orchestration tree and click a failed DAG run.
2. Click the failed task to open its log viewer.
3. Click **Ask Assistant** at the top of the log viewer.
4. The Assistant receives the last 1000 characters of the log, the task state, the task ID, and the DAG ID, along with a request to diagnose the failure.
5. The Assistant uses the `airflow-task-debugging` skill to frame the analysis, then explains what the stack trace means, what the likely root cause is, and what to fix before retrying the task.

Useful for: unblocking a teammate who does not know the codebase, handling a 3 am pager event, writing a clear incident ticket.

### Example 4: Hydra configuration question

You do not always need a button - you can just type a question in the Assistant panel:

I have a new project with a `config/` folder but my Composer shows "Not a valid Hydra config". What am I missing?

The Assistant routes to the `hydra-setup` and `noted-troubleshooting` skills, checks the expected folder structure and the defaults: list rules, and walks you through the likely fix.

### The "right panel" lifecycle

The Assistant lives in the **right panel**, which is collapsible:

- Click the speech bubble icon or **Ask Assistant** to open it.
- Click the X on the panel header to close it. The chat history is preserved across close/reopen for the lifetime of the browser session.
- Use the dropdown to switch between models without losing history.
- Use the **New Chat** button to clear the history and start fresh.

### Where to go next

- **Page 1 - Your First Project** - the starting point if you are new to noted.

- **Page 6 - Serving & Deploying Models** - the most recent feature area, a good place to ask the Assistant questions if you are exploring deployment workflows.